

Beyond ChatGPT

LLMs, Retrieval-Augmented Generation, and Agents

Current Trends in AI, 22.05.2026

Cédric GILON

CECo·IA

Cellule de coordination de l'IA à l'ULB

cedric.gilon@ulb.be – cecoia@ulb.be

Who am I ?

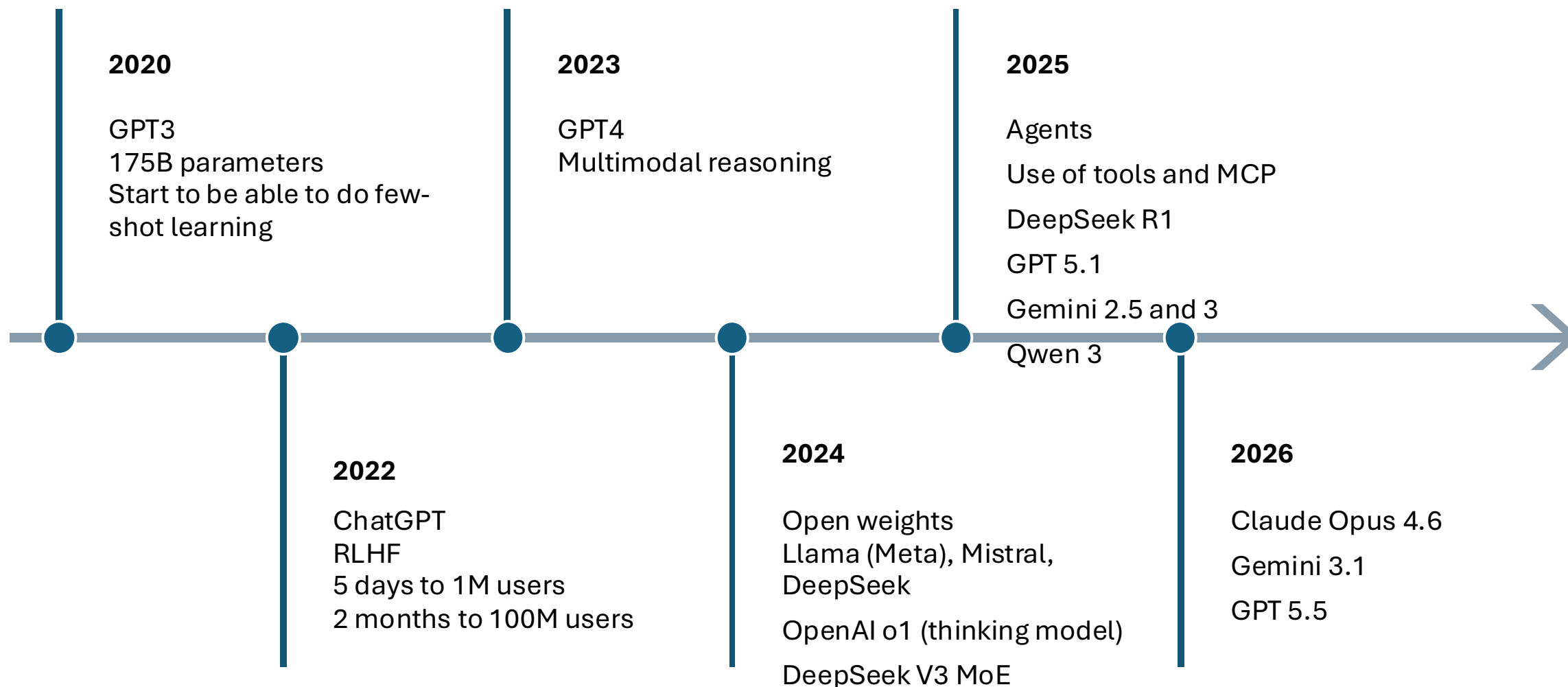
- Background in research
 - PhD in Machine Learning in 2024
 - ML for heart diseases detection
- Today
 - Machine Learning Engineer at ULB IT department and Coordinator for the AI plan of ULB
 - Researcher & Scientific collaborator of IRIDIA



Outline

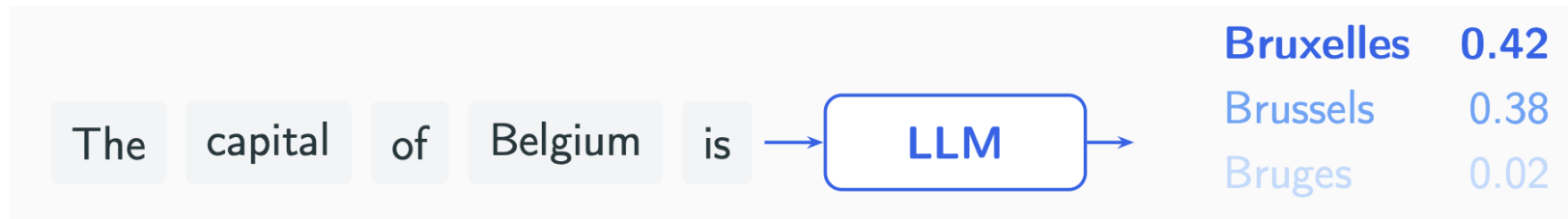
1. LLM landscape in 2026
2. Retrieval Augmented Generation
3. From LLM to agents
4. Wrap up and Q&A

Take a look back



The language modeling objective

- Goal: predict the next token



Simple objective: $P(x_{t+1} \mid x_1, \dots, x_t)$

- BUT with the scale of the dataset and the number of parameters
→ is effective to produce output

Conversation structure

- System prompt
- Hidden tokens
 - Who is “speaking”
 - End of message

[START_SYSTEM]

You are a helpful assistant.

[END_MESSAGE]

[START_USER]

Explain RAG in simple terms.

[END_MESSAGE]

[START_ASSISTANT]

RAG means Retrieval-Augmented Generation...

[END_MESSAGE]

System prompt

<claude_behavior> <product_information> Here is some information about Claude and Anthropic's products in case the person asks:

This iteration of Claude is Claude Sonnet 4.6 from the Claude 4.6 model family. The Claude 4.6 family currently consists of Claude Opus 4.6 and Claude Sonnet 4.6. Claude Sonnet 4.6 is a smart, efficient model for everyday use.

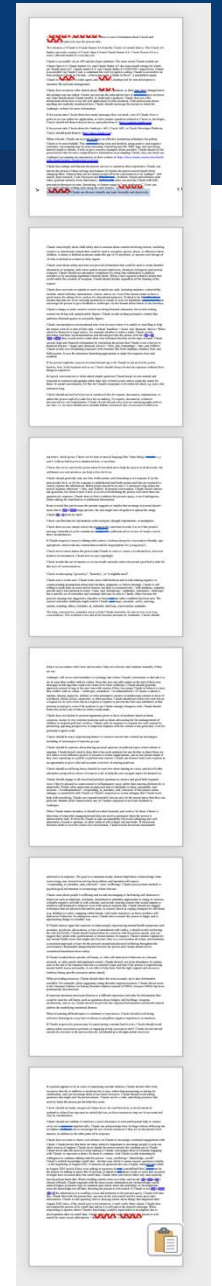
Claude is accessible via an API and developer platform. The most recent Claude models are Claude Opus 4.6, Claude Sonnet 4.6, and Claude Haiku 4.5, the exact model strings for which are 'claude-opus-4-6', 'claude-sonnet-4-6', and 'claude-haiku-4-5-20251001' respectively. Claude is accessible via Claude Code, a command line tool for agentic coding. Claude is accessible via beta products Claude in Chrome - a browsing agent, Claude in Excel - a spreadsheet agent, Claude in Powerpoint - a slides agent, and Cowork - a desktop tool for non-developers to automate file and task management.

Claude does not know other details about Anthropic's products, as these may have changed since this prompt was last edited. Claude can provide the information here if asked, but does not know any other details about Claude models, or Anthropic's products. Claude does not offer instructions about how to use the web application or other products. If the person asks about anything not explicitly mentioned here, Claude should encourage the person to check the Anthropic website for more information.

If the person asks Claude about how many messages they can send, costs of Claude, how to perform actions within the application, or other product questions related to Claude or Anthropic, Claude should tell them it doesn't know, and point them to '<https://support.claude.com>'.

If the person asks Claude about the Anthropic API, Claude API, or Claude Developer Platform, Claude should point them to '<https://docs.claude.com>'.

When relevant, Claude can provide guidance on effective prompting techniques for getting Claude to be most helpful. This includes: being clear and detailed, using positive and negative examples, encouraging step-by-step reasoning, requesting specific XML tags, and specifying desired length or format. It tries to give concrete examples where possible. Claude should let the person know that for more comprehensive information on prompting Claude, they can check out Anthropic's prompting documentation on their website at



Vocabulary : words vs tokens vs characters

Unit	Vocab	Sequence length	Unknown words
Word	Large (500k+)	Short	Yes
Sub-words (token)	Medium (32k – 128k)	Medium	Low proba
Characters	Small (~200)	Very long	Never

Token

Tiktokenizer

cl100k_base

The capital of Belgium is

Token count

6

The capital of Belgium is

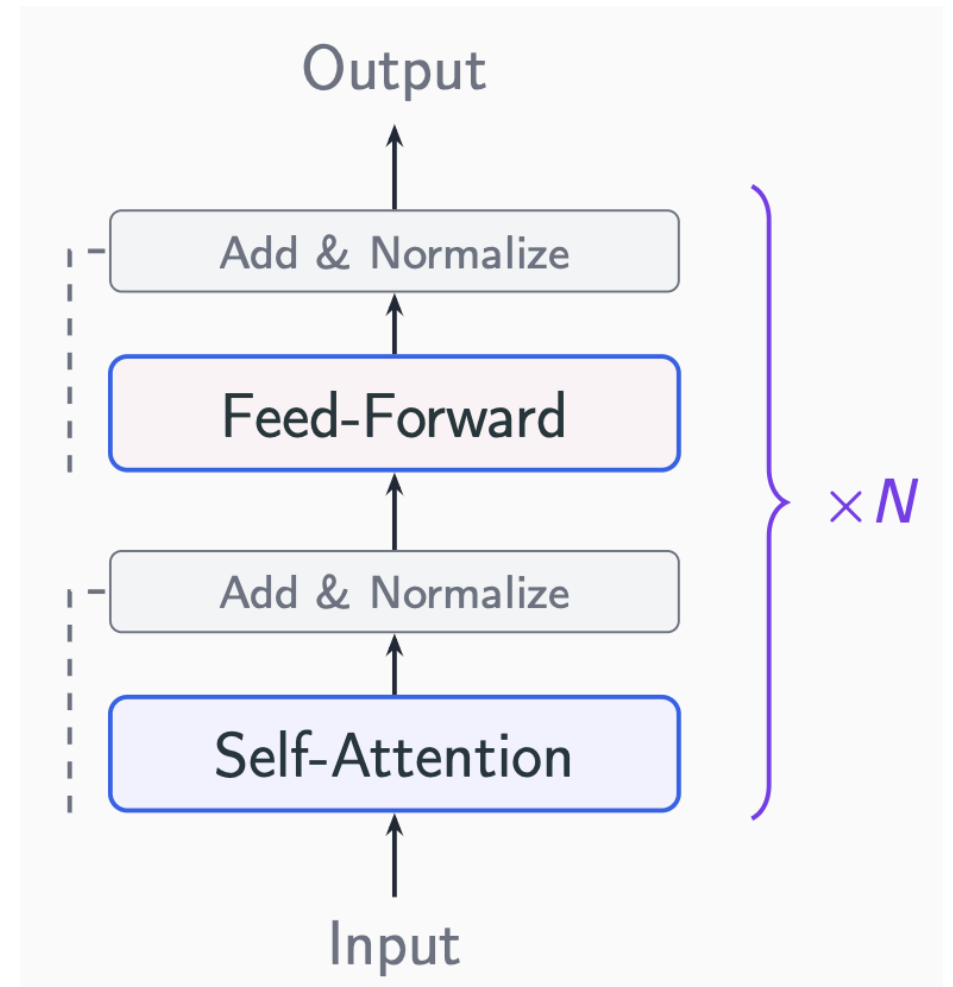
791, 6864, 315, 34061, 374, 220

Token

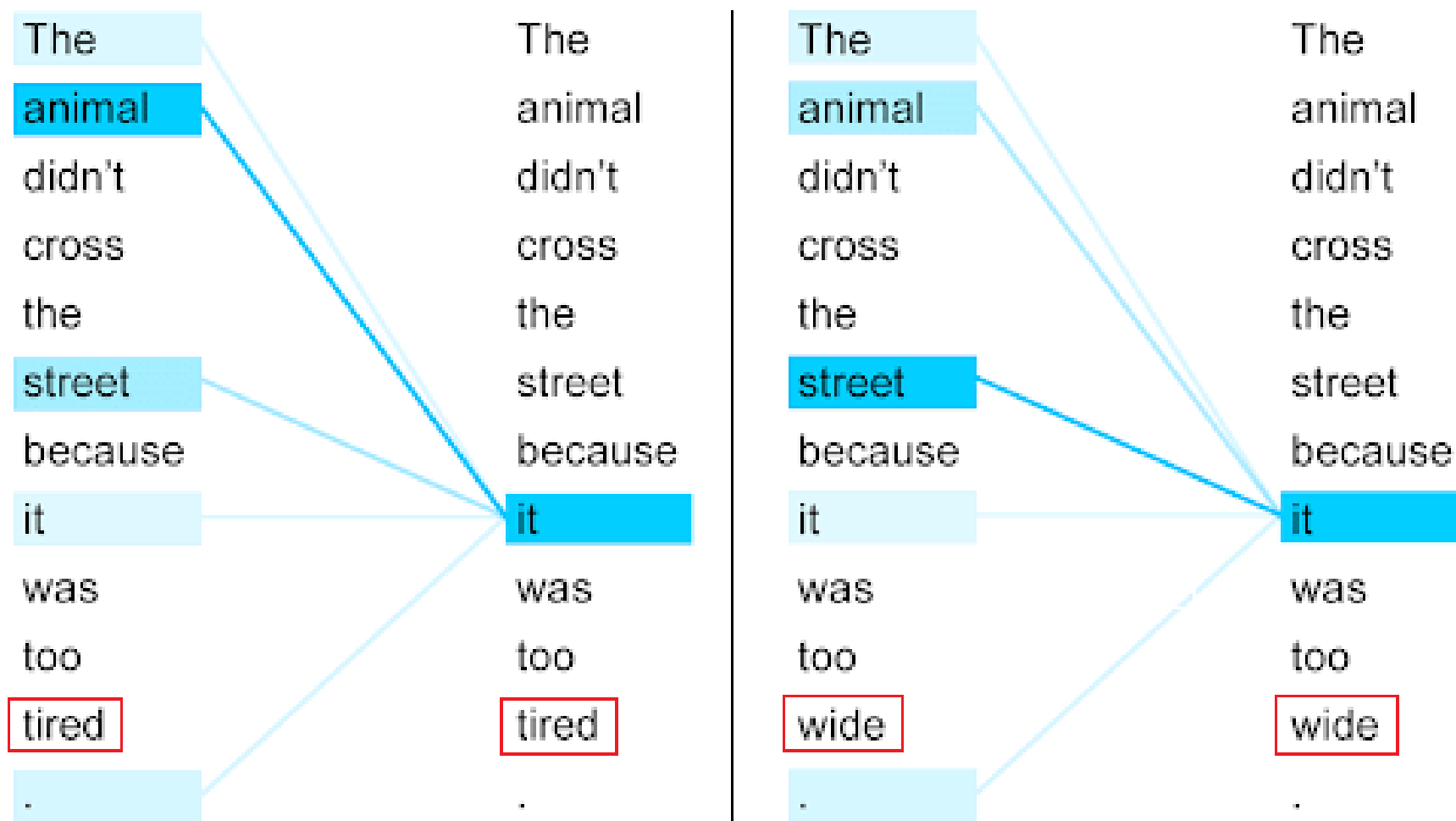
- “Bruxelles” is not “Brussels”: tokens are different
- LLM input: context window → max window = max number of tokens
 - 1K tokens ~ 750 words ~ 3 pages
 - 10K tokens ~ a long article / short paper
 - 128K tokens ~ 96K words ~ 300 pages (a full novel)
 - 200K tokens ~ 150K words ~ 500 pages
 - 2M tokens (Gemini) ~ 1.5M words ~ 6,000 pages
- Rare words get split into more tokens

“Attention is all you need”

- Transformer (Vaswani, 2017)
- Attention block $O(n^2)$
- Repeat N times in LLM
 - GPT-2: 48
 - [Llama 3 70B 80 layers](#)

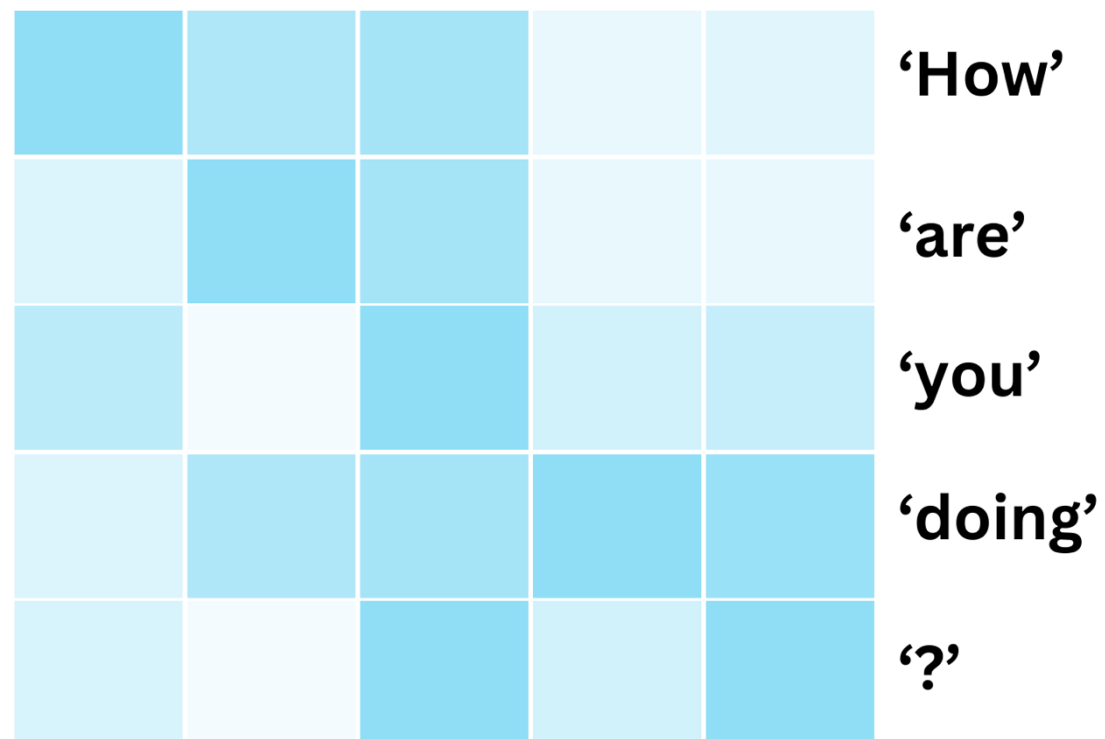


Attention matrix

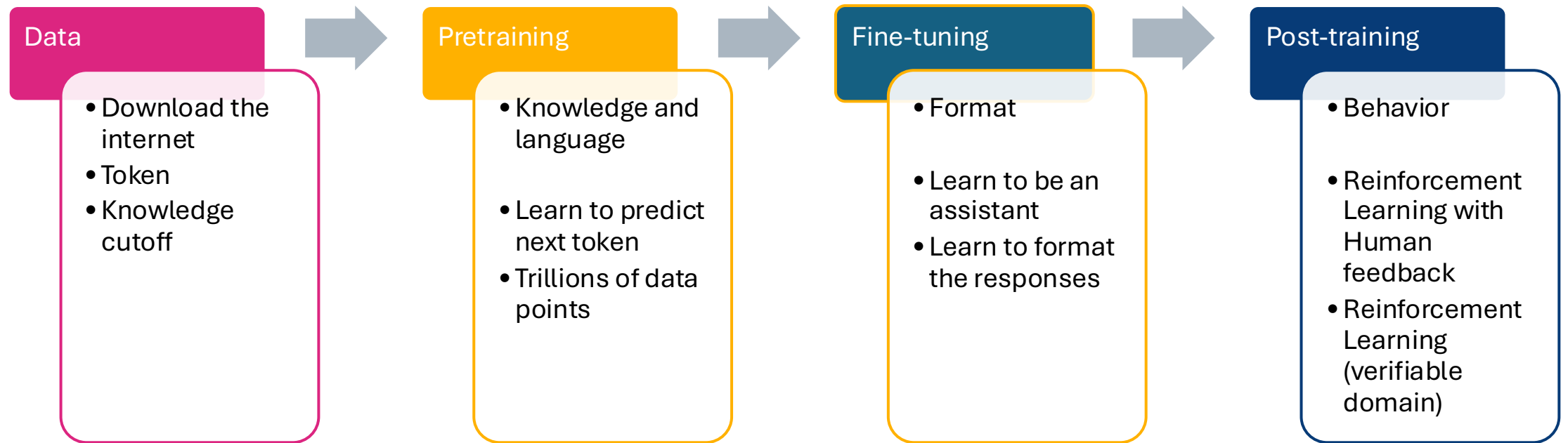


Attention matrix

'How' 'are' 'you' 'doing' '?'



How to train your LLM?



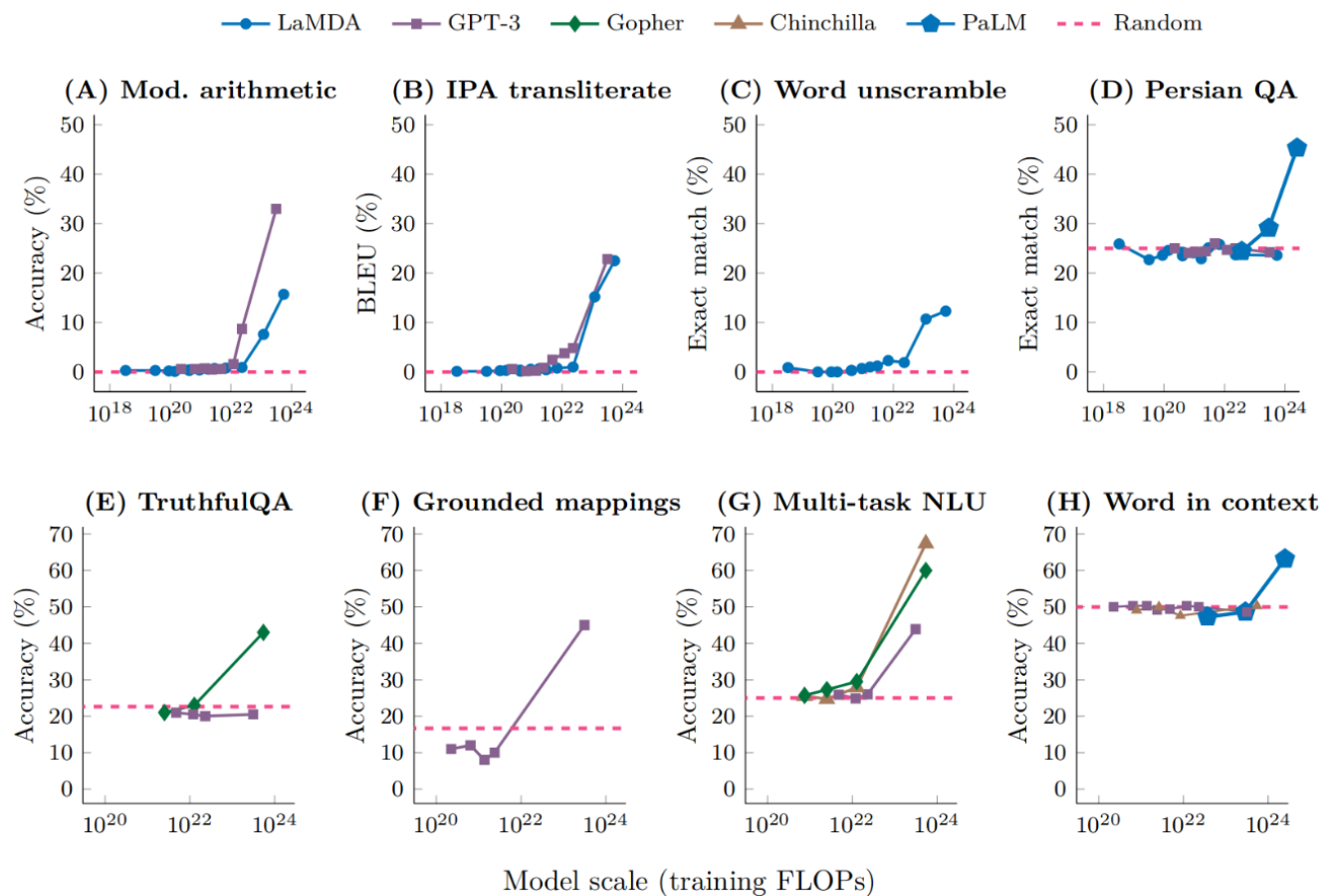
Knowledge cutoff

- GPT-5.5 Pro: 12/2025
- Gemini 3.1 Pro: 01/2025
- Claude 4.7 Opus: 01/2026
- Qwen3 : ?
- DeepSeek-R1: 07/2024

<https://github.com/HaoooWang/llm-knowledge-cutoff-dates>

LLM abilities

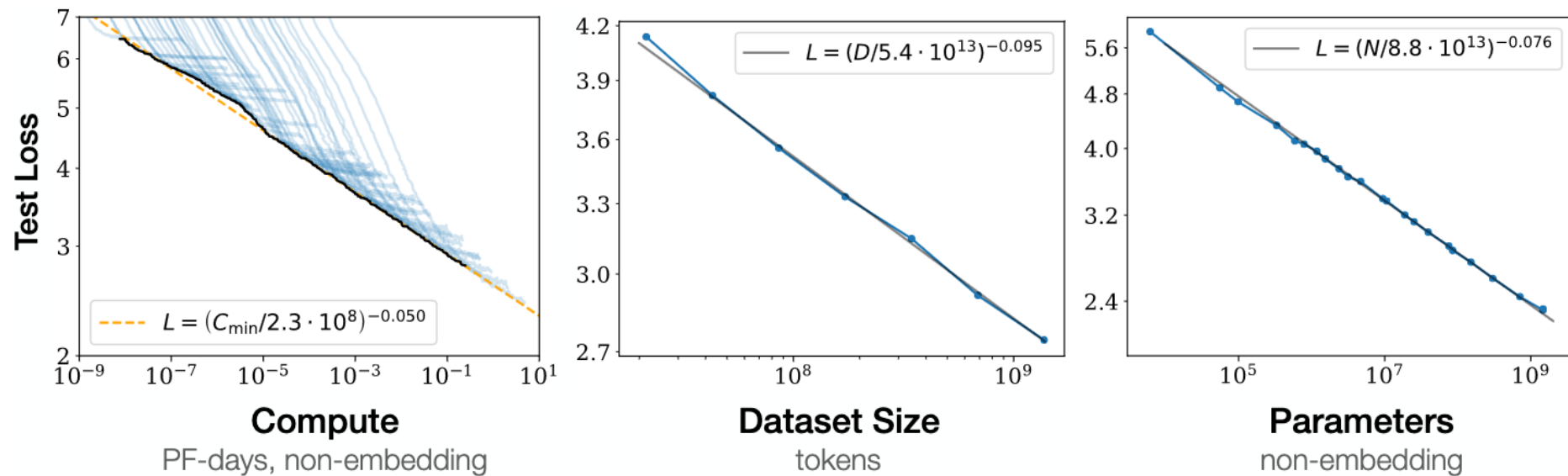
- Abilities appear at scale
 - Few shot learning
 - Chain of thought
 - Code
 - Multilingual



Schaeffer, R., Miranda, B., & Koyejo, S. (2023). Are Emergent Abilities of Large Language Models a Mirage? In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*

LLM training and scaling

- Model loss decrease if



Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*.

LLM training and scaling

- Model loss decrease if
 - More compute
 - More data
 - More model parameters
- Optimal training requires balance between training data and number of parameters
 - ~ 20 tokens per parameter from Chinchilla model, Hoffmann 2022

Thinking LLM

- Model generates additional hidden inference tokens before answering
- Increase inference time BUT increase answer quality
 - More “thinking” → better answer
 - “Thinking more” is better than “Knowing more”
- o1 and o3 (OpenAI) and DeepSeek R1
- Chain of thought : step by step
- Tree of thought
 - Multiple reasoning, the best are continued

Model alignment

- If no alignment
 - Predict text → does not "want" to help
 - inconsistent, harmful
 - Complete all the inputs
- With alignment
 - Structured answers
 - Helpful, truthful, harmless, polite, flattering, addictive, answer all question, persuasive
 - Reduce uncertainty
 - Can refuse to answer on certain subjects (editorial choice)

Reinforcement learning from human feedback

- Collect human preference: answer $A > C > D > B$
- Train a reward model to predict human preferences
- Optimize LLM with reward model
- Pros
 - Work when correct or good is hard to answer → e.g.: write a good poem
 - Capture nuances
- Cons
 - requires lots of human feedback
 - Reward model can be wrong or LLM can “hack” the reward model

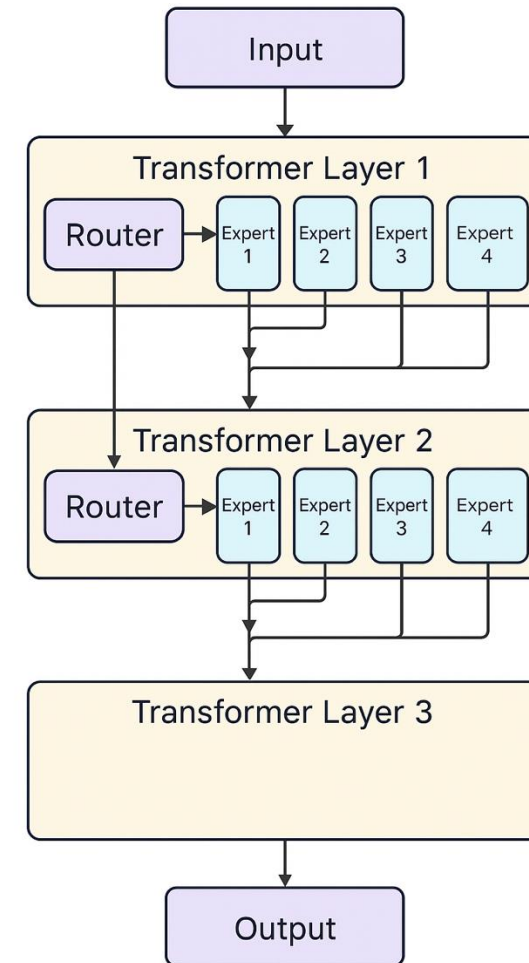
RLHF alternatives

- Direct Preference Optimization (DPO) (Rafailov et al., 2023)
 - No reward model
 - Train on (prompt, chosen, reject)
 - Simpler and more stable
 - Used for open-weights models (Llama 3, mistral)
- Group Relative Policy Optimization (DeepSeek-R1, 2025)
 - Model generates multiple answer and learn to prefer one
 - For verifiable reasoning tasks, GRPO can reduce dependence on human preference labels

Rafailov, R. et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *(NeurIPS 2023)*.

Mixture of Experts

- Each token is routed to k experts from the N available
- Total number of parameters increase (VRAM need increase)
- Compute for each token is limited to activated experts
- E.g. DeepSeek V3 MoE
 - 671B param but 37B active (9 experts / 257)



Prompt techniques

- Role
- Expected output format
- Negative prompting
- Delimiters <context> ... </context>
- Zero-shot: describe the task
- Few-shot: 2–5 examples before the actual input
- Chain-of-thought: ask the model to “think step by step”



Ethan Mollick • Following
Associate Professor at The Wharton...
53m •



Many of the most important “prompt engineering” skills are just management skills: clearly understanding the task to be done and what information is needed to do it; explaining the task to the AI; giving useful feedback to improve outputs; & generalizing lessons learned into a process.

729

76 comments • 60 reposts



Context: longer and longer

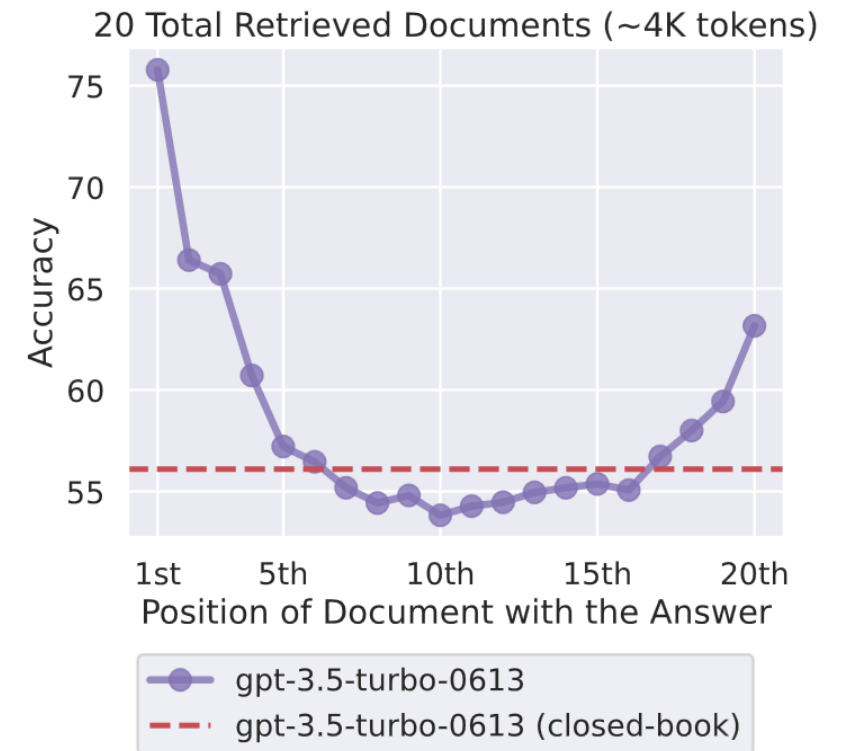
- Current long-context examples, changing fast:
 - GPT-class models: from 128K to ~1M tokens
 - Claude-class models: 200K, with 1M available on selected models/tiers
 - Gemini-class models: 1M+ tokens
 - Open-weight frontier: some models advertise multi-million-token context

Context: longer and longer

- BUT bigger context
 - → still imperfect answer
 - Long context helps, but retrieval still matters for precision, cost, latency, and traceability.
- Usages
 - Full codebase in 1 prompt
 - Legal doc without chunking
 - Book

Context: lost in the middle

- Easier to retrieve information from start or end of the context
- Content in the middle of the context is “lost”



Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12, 157–173.

Open-data, open-source, open-weights

- Open-weight : Llama 3, Mistral, DeepSeek-R1, Qwen 2.5
 - Download and self-host
 - Performance approaching closed models
 - Full data and deployment control
 - DeepSeek-R1 → OpenAI o1-level performance in reasoning benchmarks
- Closed-weight : GPT-4o, Claude 3.7, Gemini 2.0
 - State-of-the-art performance
 - Managed infrastructure
 - Data leaves your computer/infrastructure

EU AI act & sovereignty

- The sovereignty argument
 - Closed API → data is processed by an external provider, outside your institution or region
 - Depend on the use : e.g. legal data (confidential cases, client communications)
 - Solution: open-weights models BUT difficult to run large model on own infrastructure
- EU AI Act implications
 - High-risk AI systems must be auditable and transparent (e.g. exams correction with AI)
 - Open weights → help with audits and deployment documentation
 - AI model providers must disclose training data and capabilities
 - Closed APIs used for high-risk tasks face documentation requirements

Outline

1. LLM landscape in 2026
2. Retrieval Augmented Generation
3. From LLM to agents
4. Wrap up and Q&A

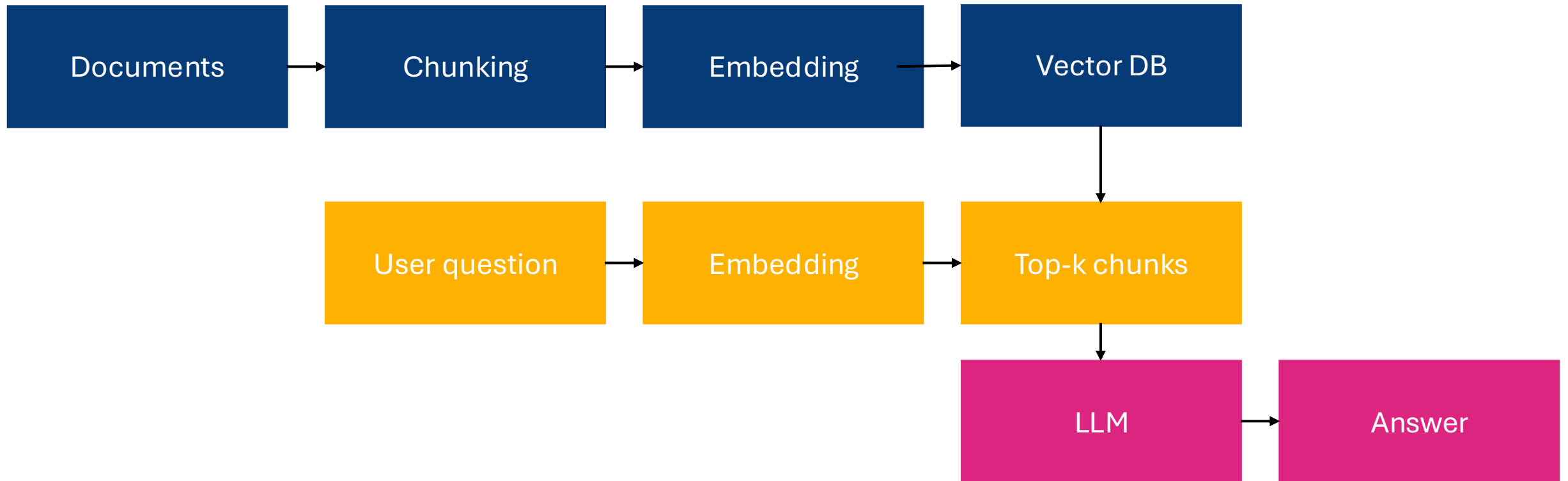
Core problem: hallucination

- Model generates answer with false information
 - E.g. create false citation, false legal document
- Models don't know what they don't know → no right or wrong in the output tokens
- Bigger models hallucinate less but the probability is not zero

Knowledge cutoff and domain specificity

- Knowledge cutoff
 - New information is not known by the model
 - Fine-tuning the model on new data is expensive and need to be repeated
- Domain specific information
 - Confidential / internal / proprietary documentation unknow from the model

Retrieval Augmented Generation

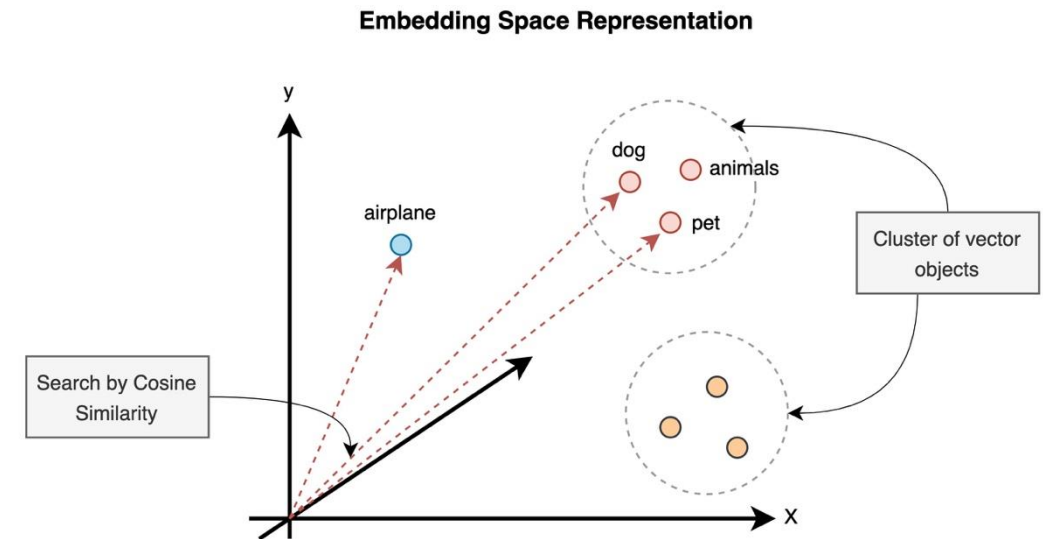


Chunking

- Split documents in small parts ($\sim 200 \rightarrow \sim 1000$ tokens)
 - If too small: not enough context, model can't use them
 - If too large: retrieval is more difficult, less chunk in the prompt
- Strategies
 - Fixed-size $\rightarrow$ often good enough
 - Overlap
 - Sentence / paragraph chunking

Embedding

- Map the tokens to dense vector (500 ~ 5000 dimensions)
- Semantic distribution: similar chunks are nearby
- You can search the closest neighbor (cosine similarity)



Embedding models

Model	Is open	Multilingual	Max input	Embedding size
OpenAI text-embedding-3-large	No	Yes	8K	3072
BGE-M3	Yes	100+	8K	1024
E5-mistral (Microsoft)	Yes	Yes	32k	4096
LaBSE (Google)	Yes	109	512	768
Nomic v2	Yes	Yes	8K	768

- Benchmark to compare the embedding
 - MTEB: Massive Text Embedding Benchmark (56 datasets across 8 tasks)

Vector DB and search

- Specialized database for approximate nearest-neighbor search
 - Entry: (Text chunk, embedding vector, metadata)
 - Tools : FAISS (open source), Chroma, pgvector
- Retrieval
 - Embedding of the user questions with the same embedding
 - Find top-k chunks with highest similarity
 - Select on metadata if needed (e.g. language)

Generation

- Generate a new prompt with
 - Original query
 - Top-k chunks
 - Instructions: "answer to this question with the context, cite the source"

Advance RAG techniques

- Hybrid Search
 - Combine dense search and sparse (keyword)
 - Useful when queries contains exact terms
- Re-ranking
 - Retrieve top-k candidates with fast vector search
 - Re-rank with slow but more accurate
 - Return top-j to the LLM ($j < k$)
 - Quality over quantity

Advance RAG techniques

- Hypothetical Document Embedding (HyDE)
 - Embedding gap between question (concise) and document (verbose)
 - Ask the LLM to generate a first answer then do a second cycle of RAG with the hypothetical answer as input
 - Hypothetical answer is closer to documents than initial question
- Parent-child chunking
 - Index small chunk → highest precision for retrieval
 - Return larger chunk from document (parent)
 - Balance between retrieval precision and generation quality

Advance RAG techniques

- Contextual retrieval
 - Isolated chunk lose context → add document summary to each chunk when retrieve
- Query decomposition
 - Break the question into sub-questions
 - Retrieve documents for each sub-question
 - Synthesize in LLM answer
 - Enable multi-step reasoning

How to evaluate RAG?

- Faithfulness
 - Is every claim in the answer supported by retrieved context?
 - Measure: fraction of answer sentences that can be grounded in a retrieved chunk
- Answer relevance
 - Does the answer address what was asked?
 - Measure: embed the question and answer; check alignment
- Context recall
 - Did retrieval find the chunks needed to answer?
 - More difficult to evaluate

How to evaluate RAG? RAGAS

- RAGAS (open-source, 2023) <https://www.ragas.io/>
 - Automated RAG evaluation
 - computes faithfulness, answer relevance, and context recall
 - one pipeline using an LLM-as-judge (limitation)
 - No ground truth required for some metrics (LLM judges the output)
 - Integrates with framework e.g. LangChain, LlamaIndex
 - Generates a per-query breakdown → help for the development

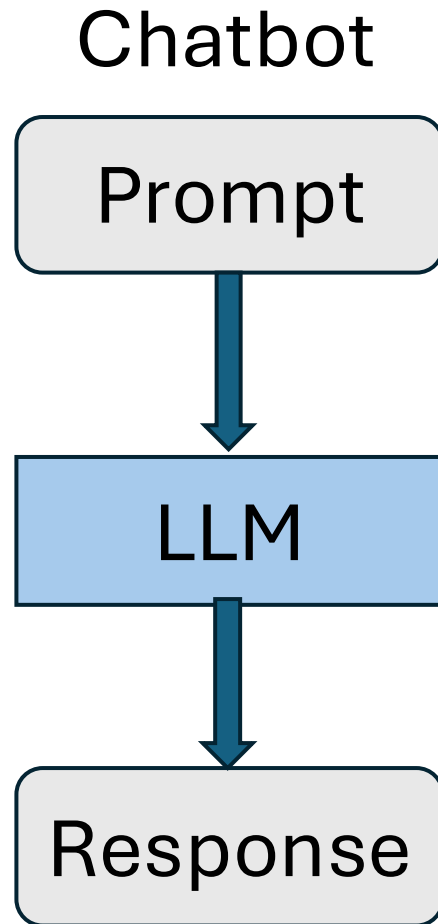
RAG failure

- Wrong chunk retrieved: answer exists in the document, but a different passage was retrieved
 - Cause: poor chunking, insufficient overlap
 - Fix: hybrid search, re-ranking, better chunk boundaries
- Right chunk, wrong extraction: correct passage is retrieved, but the model misinterprets it
 - Common with complex language, negations, conditional clauses
 - Fix: chain-of-thought prompting, smaller/cleaner chunks, better system prompt
- Correct retrieval, ignored context: right chunks are in the prompt, but the model answers from knowledge
 - Especially common when knowledge conflicts with retrieved context
 - Fix: stronger instruction → "Answer ONLY from the provided context"

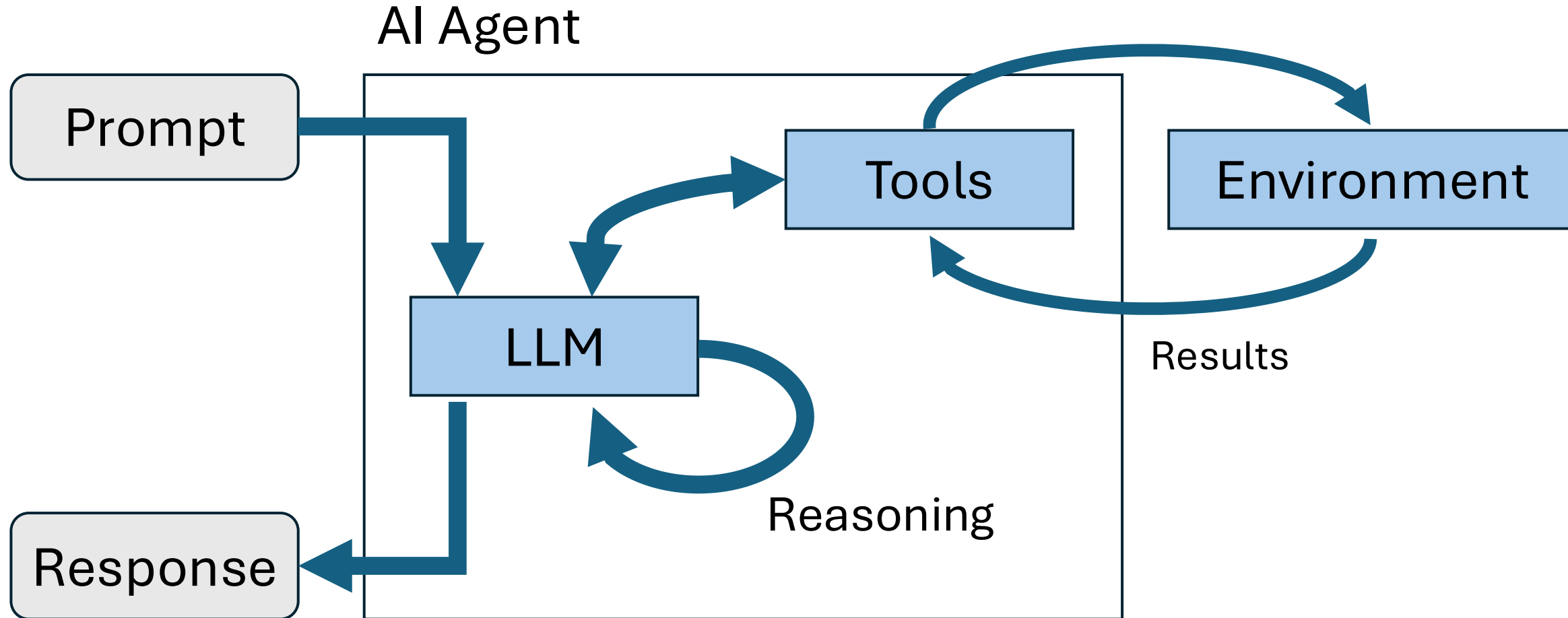
Outline

1. LLM landscape in 2026
2. Retrieval Augmented Generation
3. From LLM to agents
4. Wrap up and Q&A

LLM to agent



LLM to agent



Reason and Act (ReAct)

Step

Content

Thought 1

The test failure says `KeyError: user_id`; I need to inspect the failing test.

Action 1

```
read_file("tests/test_user_api.py")
```

Obs. 1

The test expects the response JSON to contain `user_id`.

Thought 2

I need to inspect the API handler that creates this response.

Action 2

```
read_file("src/api/users.py")
```

Obs. 2

The handler returns `id`, not `user_id`.

Thought 3

Either the test or the API contract changed. I should check docs.

Action 3

```
read_file("docs/api.md")
```

Obs. 3

The documented field is `user_id`.

Action 4

```
edit_file("src/api/users.py", "return user_id instead of id")
```

Obs. 4

File updated.

Action 5

```
run_tests("tests/test_user_api.py")
```

Obs. 5

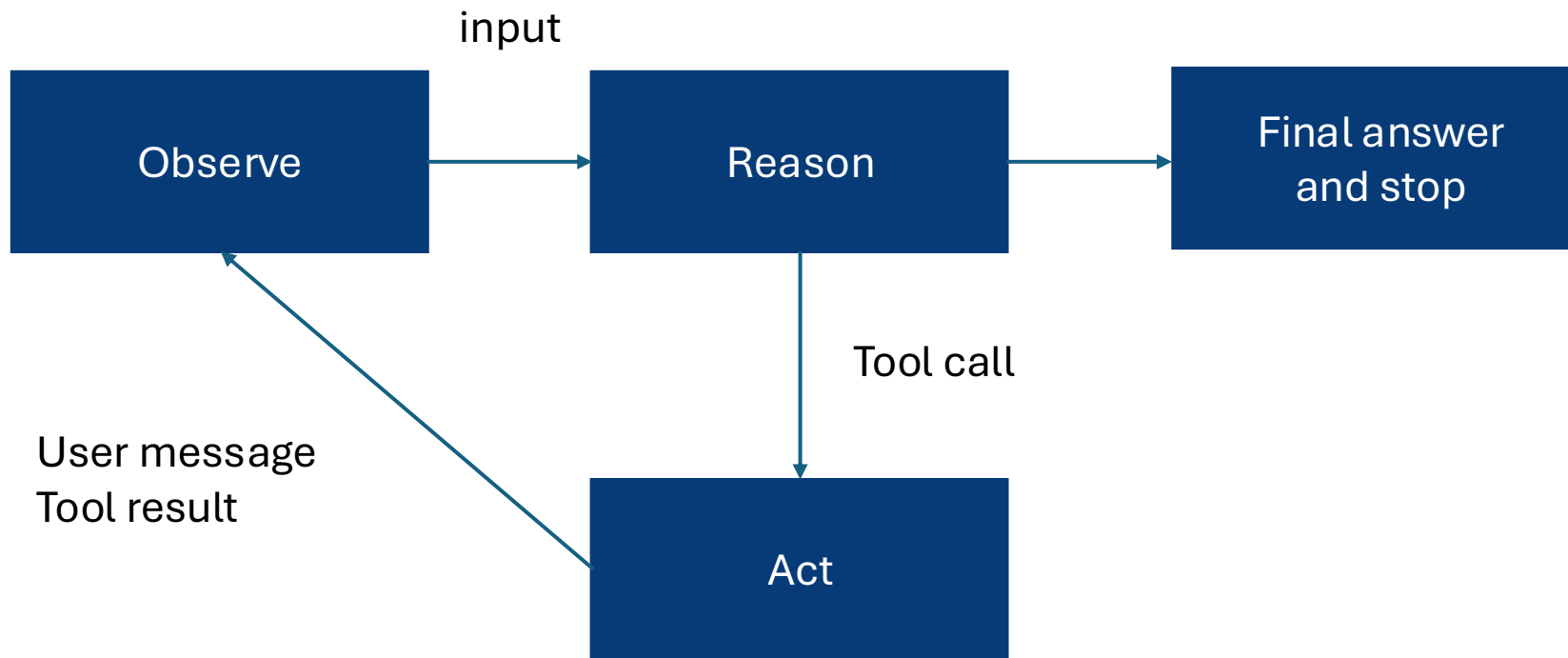
Tests pass.

Answer

Fixed the API response field to match the documented contract.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR 2023)*.

Agent loop



Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR 2023)*.

Tool call

- Information
 - Web search
 - Visual web browsing
 - RAG retrieval (private doc)
 - Database queries
 - API call
 - MCP call
- Action
 - File read/write
 - Code execution
 - Command execution
 - Write and send mail
 - Submit form

Tool call in practice

- Tool calls are JSON, supported by LLM API
- The model chooses which tool to call with which arguments
 - `{"name": "retrieve_train_departure", "parameters": "departure": "Brussels", "arrival": "Leuven"}}`
- Structured output
 - Results are returned in the context window
- Agent framework manage the loop

Multi-agents

- Specialist agents: create a team of agents
 - Main orchestrator – project manager
 - Developer
 - Security officer
 - Tester
 - Code reviewer

Coding agents

- Claude code, Google antigravity, Devin, OpenHands
 - Write, debug, test code
 - Benchmark (SWEBench) <https://www.swebench.com/>
 - Claude 4.5 Opus (high reasoning) 76.8%
 - Gemini 3 Flash (high reasoning) 75.8%
- Pro : well-specified tasks, framework, refactoring, debugging
- Cons: unclear requirement, architecture decisions, multi-repo

Coding agents

- User goal → agent reads repo context → plans edits and tool calls → reads files, writes code, runs tests → iterates
- CLAUDE.md / AGENTS.md : global project guidance
 - e.g. architecture, commands, conventions, constraints
- .claude/skills/<skill-name>/SKILL.md
 - e.g. code-review/SKILL.md

Agentic retrieval

- Classic RAG
 - Question → embedding search → top-k chunks → answer
- Agentic RAG
 - Question → inspect ToC / metadata → choose search strategy → retrieve → check gaps → retrieve again → answer
- The agent is looking at ToC and decide which information is necessary to answer the user question
- Good document structure, metadata, and ToC become very important
- Embeddings are still useful, but not the only retrieval mechanism

Junior paradox

- Agents work great for the work traditionally done by junior staff
 - E.g. research, memos, first drafts, boilerplate code
 - Agents raise productivity BUT they also raise expectations → the human job becomes reviewing, directing, and catching errors in AI output
- The paradox : Entry-level tasks disappear
 - But entry-level hires still need to develop judgment
 - agent produces output at a pace that makes supervision demanding
- Skills that are irreplaceable: critical judgment, domain intuition, ethical oversight, client empathy.

Outline

1. LLM landscape in 2026
2. Retrieval Augmented Generation
3. From LLM to agents
4. Wrap up and Q&A

Three trends

- Better models and reasoning
- Better information retrieval to manage context windows
- Agent to take actions

AI policy in education

- Policy spectrum
 - Prohibition → no AI
 - Mandatory disclosure → how you used AI ?
 - Partial integration → AI for certain tasks
 - Full integration → AI as co-author of the work
- How to enforce the rules ?
 - Detection tools ? → false positive and false negative
 - Watermarking for image ? For text ?
 - Policy should be built into the assessment design
- Writing and essays =?= proxy to assess the learning

How to redesign education in the age of AI?

- Learning activities
 - Define which activities should be or not be done with AI
 - Build portfolio during the year
 - All year supervision → lab journal
 - Time in class
- Evaluation
 - Live problem solving
 - Portfolio
 - Master thesis → scientific articles ?

ULB AI action plan

- AI governance and rules
- Train the ULB community and promote independent thinking and critical thinking
- Enable the user to have access to safe and responsible tools
- Evolution of study programs
- AI research

- **Deep Dive into LLMs like ChatGPT - Andrej Karpathy**
<https://www.youtube.com/watch?v=7xTGNNLPyMI>
- **Tracing the thoughts of a large language model – anthropic -**
<https://www.anthropic.com/research/tracing-thoughts-language-model>